WEEK 6 ASSIGNMENT

Custom Nmap NSE

Security Auditing Toolkit

Student Name	Patel Mantra H.
Batch	Batch-05
Course	Cyber Security
Total Marks	100

Project overview

THE PROBLEM

Manual service checks are repetitive, inconsistent, and difficult to turn into a clear assessment report.

• Identify exposed service details

• Review selected HTTP security headers

• Flag observable configuration concerns

• Export consistent findings for review

PROJECT GOAL

Build a small collection of original Nmap Scripting Engine (NSE) scripts and a repeatable workflow for authorized lab systems.

Deliverables

• Two custom NSE scripts

• Runbook and safety scope

• Sample structured findings

• One-page assessment summary

Scope, ethics & lab setup

IN SCOPE

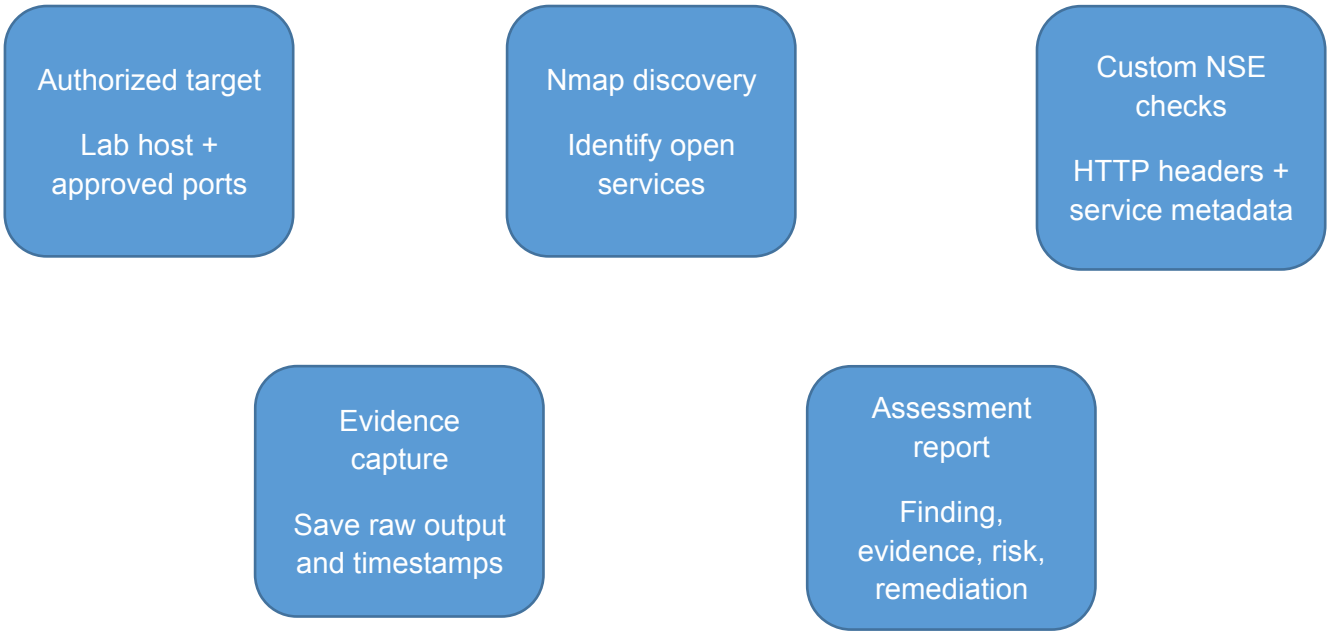
• Only localhost, an isolated VM, or systems with explicit written permission

• Non-destructive service enumeration and HTTP response-header review

• No exploit attempts, credential guessing, stealth, or denial-of-service tests

- Record the target, date, command, output, and limitations

Toolkit architecture



Custom NSE script 01 — HTTP security headers

Purpose

Request a page and report whether selected defensive headers are present. Missing headers are review items, not proof of exploitability.

http-security-headers.nse

```
local http = require "http"

local shortport = require "shortport"

local stdnse = require "stdnse"

description = [[Checks common HTTP security headers on an authorized lab web service.]]

author = "Project 10 student"

license = "Same as Nmap"
```

```
categories = {"safe", "discovery"}

portrule = shortport.http

action = function(host, port)
  local r = http.get(host, port, "/")
  if not r then
    return "No HTTP response received"
  end
  local headers = r.header or {}
  local wanted = {
    "strict-transport-security",
    "content-security-policy",
    "x-content-type-options",
    "x-frame-options"
  }
  local out = {"HTTP security header review:"}
  for _, name in ipairs(wanted) do
    if headers[name] then
      table.insert(out, name .. ": present")
    else
      table.insert(out, name .. ": missing")
    end
  end
  return stdnse.format_output(true, out)
end
```

Output: present/missing status for four headers; verify application requirements before assigning severity.

Custom NSE script 02 — HTTP server banner

Purpose

Collect the HTTP Server header, if disclosed, so an assessor can review unnecessary product/version exposure.

http-server-banner-review.nse

```
local http = require "http"

local shortport = require "shortport"

description = [[Reports the HTTP Server response header when available.]]

author = "Project 10 student"

license = "Same as Nmap"

categories = {"safe", "discovery"}

portrule = shortport.http

action = function(host, port)
  local r = http.get(host, port, "/")
  if not r then
    return "No HTTP response received"
  end
  local server = (r.header or {}).server
  if server then
    return "HTTP Server header disclosed: " .. server
  end
  return "No HTTP Server header disclosed"
end
```

Installation & safe execution

1 Save the scripts

Place the .nse files in a working directory, then load them explicitly with --script. Avoid installing unreviewed scripts system-wide.

2 Check syntax / availability

Use a current Nmap release and confirm the local Lua/NSE environment is compatible. Review each script before running it.

Example: localhost lab only

```
# HTTP service check on your own lab

nmap -sV -p 80,8080 \

  --script ./http-security-headers.nse \

  127.0.0.1

# Save output for evidence

nmap -sV -p 80,8080 \

  --script ./http-server-banner-review.nse \

  -oN project10-results.txt 127.0.0.1
```

Structured assessment report

Finding	HTTP security headers missing
Evidence	Exact header names + target + timestamp
Risk	Context-dependent; review application and browser controls
Recommendation	Set appropriate headers at the web server / reverse proxy

Testing, validation & limitations

VALIDATION PLAN

- Run against a local web server with known headers
- Compare results with browser developer tools or curl
- Test a response with no Server header
- Keep raw Nmap output for reproducibility
- Repeat after configuration changes

LIMITATIONS

- HTTPS/TLS and virtual-host behavior may need separate handling
- A GET request to / may not represent every application route
- Header presence does not prove the configuration is correct
- Banner strings can be hidden, modified, or misleading

- Scripts are a starting point, not a replacement for expert review

One-page project report template

Project Title	Custom Nmap NSE Security Auditing Toolkit
Why this project?	To automate repeatable, low-impact checks on authorized lab web services.
Tools	Nmap, NSE/Lua, local test web server; optional curl/Python for validation.
Methodology	Defined scope → ran service discovery → executed custom checks → saved evidence → reviewed findings.
What I learned	NSE script structure, service enumeration, HTTP headers, evidence handling, report writing.
Challenges & fixes	Validated header key handling and tested against known local responses; documented script limitations.
Final outcome	Two read-only scripts and a repeatable reporting workflow; results depend on the tested lab configuration.